16

Лекция 22–23 декабря

Сжатие и границы

Константин Чухарев

Дискретная математика — Осень 2026

Код Хаффмана и границы

> *Основная задача связи состоит в точном или приближенном воспроизведении в некотором месте сообщения, выбранного для передачи в другом месте.*
>
> ***— Клод Шеннон***

Защита и сжатие

Код Хэмминга защищает от ошибок, добавляя избыточность.

Код Хаффмана решает обратную задачу — сжать данные до минимального размера без потерь.

> Частым символам — короткие кодовые слова, редким — длинные.

Азбука Морзе делает то же самое: буква E — одна точка, а Q — четыре знака.

> Сжатие здесь — без потерь: распакованный файл совпадает с исходным бит в бит.
>
> С потерями живут картинки и звук — у них другая норма качества и другая математика.

Префиксный код

Слова разной длины полезны, только если поток читается без пауз и возвратов. Этого добиваются одним запретом.

Определение 1 (Префиксный код)

Код называется **префиксным**, если ни одно кодовое слово не является началом другого.

Пример

Код $\{0, 10, 11\}$ префиксный: ни одно слово не начало другого.

Код $\{0, 01, 011\}$ нет: слово 0 может продолжиться как 01 или как 011 — декодер не знает, кончилось ли слово.

На дереве кода запрет звучит так: кодовые слова — листья, и ни одно не стоит на пути к другому.

Декодирование без задержки

Пример

Слова $A = 0$, $B = 10$, $C = 110$, $D = 1110$, $E = 1111$.

Поток `1101111` читается слева направо: `110` — это C, сразу за ним `1111` — это E.

Другого разбиения нет: конец слова виден в момент прочтения.

Префиксность устраняет неоднозначность: декодер не возвращается и не ждёт.

Префиксные коды называют *мгновенными* (англ. instantaneous): символ опознаётся сразу по прочтении своего слова.

Тем самым декодер работает в реальном времени — задержка ограничена длиной одного слова.

Неравенство Крафта

Не всякий набор длин годится: слово длины 1 занимает половину кодового пространства.

Теорема 1 (Неравенство Крафта)

Если бинарный префиксный код состоит из слов с длинами $l_1, \ldots, l_n$, то

$$\sum_{i=1}^{n} 2^{-l_i} \leq 1.$$

Доказательство (подсчётом строк)

Возьмём $L = \max_i l_i$.

Каждое слово продолжается до 2^{L-l_i} строк длины L, и из-за префиксности продолжения разных слов не пересекаются.

Все они среди 2^L строк, значит, $\sum_i 2^{L-l_i} \leq 2^L$ — деление на 2^L даёт неравенство.

Крафт: числа и ловушка

Пример

Код Хаффмана пяти символов $A = 0, \ldots, E = 1111$ даёт $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{16} = 1$.

Равенство означает, что кодовое дерево заполнено без дыр.

Сумма не делает код префиксным

Код $\{0, 01, 011\}$ проходит по сумме: $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} = \frac{7}{8} \leq 1$, но слово 0 — префикс слова 01.

Обратная теорема гарантирует лишь существование другого префиксного кода с теми же длинами, например $\{0, 10, 110\}$.

Код из длин: обратная теорема

Обратная теорема разрешает все наборы длин, которые проходит Крафт: длины можно выбирать раньше кода.

Теорема 2 (Обратная теорема Крафта)

Если $\sum_{i=1}^{n} 2^{-l_i} \leq 1$, то префиксный код с длинами $l_1, \ldots, l_n$ существует.

Доказательство (построением)

Упорядочим длины по возрастанию и возьмём словом i первые l_i битов двоичной записи суммы $s_i = \sum_{k<i} 2^{-l_k}$.

Корректность. Слову из l_i битов отвечает отрезок $\left[s_i, s_i + 2^{-l_i}\right)$, и отрезки идут слева направо без наложений: для $j > i$ начало s_j не меньше конца предыдущего. Вложенных отрезков нет: иначе одно слово — начало другого.

Код из длин: пример и дыры

Пример (Суммы: полная и с дырой)

Длины $(1, 2, 3, 3)$: $s_1 = 0$ даёт 0, $s_2 = \frac{1}{2} = 0.1_2$ даёт 10, $s_3 = \frac{3}{4} = 0.11_2$ даёт 110, $s_4 = \frac{7}{8} = 0.111_2$ даёт 111 — код $\{0, 10, 110, 111\}$, тот же, что строит Хаффман для частот $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$.

Длины $(1, 2)$ дают сумму $\frac{1}{2} + \frac{1}{4} = \frac{3}{4} < 1$: код $\{0, 10\}$ не заполняет дерево, и слово 1 остаётся свободным.

Дыру можно достроить новыми словами: строгое неравенство разрешает, но не требует заполнять всё.

Теорема Макмиллана усиливает неравенство: ему подчиняется любой однозначно декодируемый код, даже не префиксный.

Префиксность бесплатна: класс кодов уже, а лучшие длины те же.

Алгоритм Хаффмана

Алгоритм жадный и строит дерево снизу вверх.

Пример

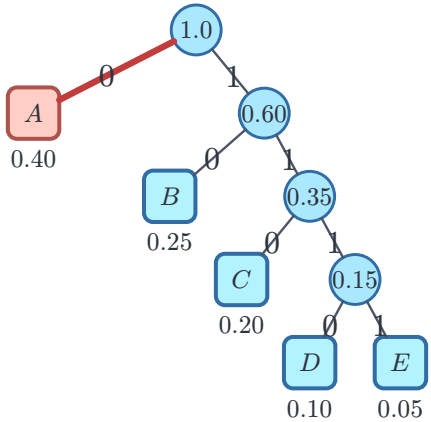
Частоты: $A = 0.40$, $B = 0.25$, $C = 0.20$, $D = 0.10$, $E = 0.05$.

1. Сливаем два символа наименьшего веса: D и E — узел 0.15.
2. Дальше C и DE — узел 0.35.
3. Дальше B и CDE — узел 0.60.
4. Наконец A и BCDE — корень 1.00.

Реализация — очередь с минимальным весом: два извлечения, одно слияние, одна вставка, а всего шагов $n - 1$.

Тяжёлые символы сливаются последними и остаются у корня — потому и получают короткие слова.

Дерево Хаффмана



Путь от корня к листу — кодовое слово: левый переход даёт 0, правый — 1.

Кодовые слова: $A = 0$, $B = 10$, $C = 110$, $D = 1110$, $E = 1111$.

Средняя длина

Определение 2 (Средняя длина кода)

$$L = \sum_i f_i l_i,$$

где f_i — частота символа, l_i — длина его слова.

Пример

$$L = 0.40 \cdot 1 + 0.25 \cdot 2 + 0.20 \cdot 3 + 0.10 \cdot 4 + 0.05 \cdot 4 = 2.10$$

бита на символ — против $\lceil \log_2 5 \rceil = 3$ битов у равномерного кода на пять символов.

При равных частотах и n — степени двойки код Хаффмана вырождается в равномерный: все длины $\log_2 n$ одинаковы, и сжимать нечего.

Выигрыш появляется, только когда частоты неравны.

Оптимальность: обменный аргумент

Теорема 3 (Обмен без ухудшения)

Среди оптимальных деревьев найдётся такое, где два редчайших символа — соседние листья максимальной глубины.

Доказательство (двумя обменами)

Пусть a, b — редчайшие символы, а x, y — соседние листья максимальной глубины оптимального дерева.

Обмен a и x меняет вклад пары на $(f_a - f_x)(l_x - l_a) \leq 0$: редчайший уходит глубже, длина не растёт.

Дерево осталось оптимальным, а символ a встал на максимальную глубину. Тем же обменом поставим b на место листа y, и пара a, b станет соседней.

Оптимальность по индукции

Теорема 4 (Оптимальность кода Хаффмана)

Для заданных частот код Хаффмана имеет минимальную среднюю длину среди всех префиксных кодов.

Доказательство (индукцией по числу символов n)

База. При $n = 1$ длина $L = 0$ — короче невозможно.

Шаг. По обменной теореме есть оптимальное дерево, где редчайшие a, b — соседи максимальной глубины.

Склейка их в лист веса $f_a + f_b$ уменьшает L ровно на $f_a + f_b$. По индукции алгоритм оптимален для $n - 1$ символов, а разворачивание листа прибавляет те же $f_a + f_b$ любому коду — дерево оптимально и для n.

Хаффман: шесть символов

Частоты: $A = 0.30$, $B = 0.25$, $C = 0.15$, $D = 0.12$, $E = 0.10$, $F = 0.08$.

Шаг	Слияние	Вес узла	Лес после шага
1	$F + E$	0.18	$0.12, 0.15, 0.18, 0.25, 0.30$
2	$D + C$	0.27	$0.18, 0.25, 0.27, 0.30$
3	$\mathrm{FE} + B$	0.43	$0.27, 0.30, 0.43$
4	$\mathrm{DC} + A$	0.57	$0.43, 0.57$
5	$\mathrm{FEB} + \mathrm{DCA}$	1.00	корень

Пять слияний собирают дерево: в каждой паре меньший вес получает ребро 0, больший — ребро 1, поэтому редчайшие F и E уходят на дно, а частые A и B поднимаются к корню.

Шесть символов: средняя длина

Символ	Частота	Код	Длина l_i
A	0.30	11	2
B	0.25	01	2
C	0.15	101	3
D	0.12	100	3
E	0.10	001	3
F	0.08	000	3

Пример

$L = 0.30 \cdot 2 + 0.25 \cdot 2 + 0.45 \cdot 3 = 2.45$ бита на символ — против 3 битов равномерного кода, экономия 18%.

Короче нельзя: теорема об оптимальности запрещает $L < 2.45$ на этих частотах.

Словарное сжатие: LZ77 и LZ78

Хаффман видит только частоты, а главный резерв реальных данных — повторения.

Словарное сжатие заменяет повтор ссылкой: канал передаёт адрес, а не копию.

Определение 3 (Тройка LZ77)

Кодер ищет совпадения в *скользящем окне* — уже закодированном хвосте входа. Повтор кодируется тройкой (o, l, a): смещение назад, длина совпадения, первый несовпавший символ.

LZ78 копит *словарь фраз* вместо окна: выход — пары из номера фразы и нового символа.

DEFLATE (gzip, zip, PNG) — LZ77 плюс код Хаффмана: LZ выучивает повторения, Хаффман ужимает остаток.

LZ77: тройки на «ABRACADABRA»

Тройка	Совпало	Свежий символ
$(0, 0, A)$	—	A
$(0, 0, B)$	—	B
$(0, 0, R)$	—	R
$(3, 1, C)$	A	C
$(2, 1, D)$	A	D
$(7, 4)$	ABRA	конец входа

Пример

Семь символов назад лежит точная копия хвоста: финальная пара $(7, 4)$ кодирует четыре буквы двумя числами. При фиксированной разрядности — 3 бита на смещение, 3 на длину, 8 на символ — пять троек и финальная пара стоят $5 \cdot 14 + 6 = 76$ бит против 88 битов ASCII.

LZ78: словарь фраз

Выход — пара: номер уже выученной фразы и новый символ.

Пара	Выдало	Новая фраза
$(0, A)$	A	$1 = \mathrm{A}$
$(0, B)$	B	$2 = \mathrm{B}$
$(0, R)$	R	$3 = \mathrm{R}$
$(1, C)$	(AC)	$4 = \mathrm{AC}$
$(1, D)$	(AD)	$5 = \mathrm{AD}$
$(1, B)$	(AB)	$6 = \mathrm{AB}$
$(3, A)$	(RA)	$7 = \mathrm{RA}$

Семь пар покрывают одиннадцать букв, и декодер строит словарь синхронно с кодером.

LZ78: вариант LZW

Пример

LZW — LZ78 с алфавитом в стартовом словаре: пара вырождается в чистый номер фразы.

На «ABRACADABRA» LZW выдаёт девять номеров по 4 бита — 36 битов против 88 исходных.

Номера коротки, потому что словарь растёт медленно: после k выученных фраз максимальный номер $|\Sigma| + k - 1$ укладывается в $\lceil \log_2(|\Sigma| + k) \rceil$ битов.

LZW жил в GIF и Unix-утилите compress, пока LZ77+Хаффман не стал стандартом DEFLATE.

Арифметическое сжатие

Хаффман округляет длину каждого символа до целого числа битов. Арифметическое кодирование округляет один раз на всё сообщение.

Определение 4 (Арифметическое кодирование)

Всё сообщение кодируется одним числом — точкой отрезка $[0, 1)$.

Символ с вероятностью p_i сжимает текущий отрезок в p_i раз, и после n символов отрезок имеет длину $W = p_1 p_2 \cdots p_n$.

Достаточно любой двоичной дроби внутри отрезка — не более $\lceil -\log_2 W \rceil$ битов.

Число битов на символ сходится к энтропии: округление длины каждого символа до целого бита, за которое платит Хаффман, исчезает.

Реальный кодер держит отрезок в 32- или 64-битных целых и отдаёт старшие биты выхода по мере их фиксации.

Арифметика: числа на слове

Пример

Для «ABRACADABRA» частотами служат сами буквы: $p_A = \frac{5}{11}$, $p_B = p_R = \frac{2}{11}$, $p_C = p_D = \frac{1}{11}$.

Длина финального отрезка $W = \left(\frac{5}{11}\right)^5 \left(\frac{2}{11}\right)^4 \left(\frac{1}{11}\right)^2$, и $-\log_2 W \approx 22.4$ бита — это nH.

Кодер выдаёт 23 бита, как и Хаффман, а вот на кривой монете $(0.9, 0.1)$ Хаффман застревает на целом бите, и только арифметика спускается к 0.47.

Арифметический кодер — блочный код Хаффмана, у которого блок вырос до всего сообщения: зазор $\frac{1}{n}$ из границы энтропии тает.

Бинарный вариант под именем CABAC работает в видеокодеках H.264 и H.265.

Шесть методов на одном слове

Сведём счёт на «ABRACADABRA»: исходно $11 \cdot 8 = 88$ битов.

Метод	Биты	Доля	Что использует
ASCII без сжатия	88	100%	ничего
Код Хаффмана	23	26%	неравные частоты
Арифметическое кодирование	23	26%	частоты без округления
LZ77	76	86%	ссылки в окно
LZ78	77	88%	словарь фраз
LZW	36	41%	только номера фраз

Тройки LZ77 стоят $3 + 3 + 8$ битов, пары LZ78 — $3 + 8$, номера LZW — 4.

Сжимаемость строк

Теорема 5 (Почти все строки несжимаемы)

Доля строк длины n, сжимаемых хотя бы на k битов, не превосходит 2^{1-k} при $k \leq n$.

Доказательство (подсчётом)

Распаковщик взаимно однозначен: две сжатые строки не разворачиваются в одну.

Сжимаемые строки — образы строк длины до $n-k$, а таких строк $2^{n-k+1}-1$.

Доля не превышает $\frac{2^{n-k+1}-1}{2^n} < 2^{1-k}$.

Пример

При $k = 10$ сжимается меньше $2^{-9} \approx 0.2\%$ строк — один файл из пятисот, а реальные тексты сжимаются в разы: они из редкого структурированного меньшинства.

Универсальный архиватор

Архиватор всего

Программа, сжимающая любой файл хотя бы на бит без потерь, невозможна по принципу Дирихле: строк длины меньше n всего $2^n - 1$, и на все 2^n строк их не хватает.

На случайных данных честный архиватор лишь растит файл — повторений нет, а заголовок добавляется.

Реальные данные сжимаемы, потому что люди порождают крошечную долю всех строк: частоты символов неравны, а куски повторяются.

Хорошо сжатый файл выглядит как случайные биты — это признак того, что структура исчерпана.

Кодер Хаффмана на Python

Вся механика — куча с минимальным весом: извлекаем два лёгких поддерева, дописываем им по биту, возвращаем слияние.

```
import heapq

def huffman(freq):
    heap = [[w, [ch, ""]] for ch, w in sorted(freq.items())]
    heapq.heapify(heap)
    while len(heap) > 1:
        lo, hi = heapq.heappop(heap), heapq.heappop(heap)
        for p in lo[1:]: p[1] = "0" + p[1]
        for p in hi[1:]: p[1] = "1" + p[1]
        heapq.heappush(heap, [lo[0] + hi[0]] + lo[1:] + hi[1:])
    return dict(heap[0][1:])
```

Запуск кодера

Драйвер считает частоты, сортирует символы по убыванию частоты и печатает таблицу кодов.

```
from collections import Counter

text = "ABRACADABRA"
freq = Counter(text)
code = huffman(freq)
for ch in sorted(freq, key=freq.get, reverse=True):
    print(ch, freq[ch], code[ch])
print("total:", sum(freq[ch] * len(code[ch]) for ch in freq), "bits vs", 8 *
len(text))
```

Выход кодера

Пример

```
A 5 0
B 2 110
R 2 10
C 1 1110
D 1 1111
total: 23 bits vs 88
```

Сумма длин даёт $5 \cdot 1 + 2 \cdot 3 + 2 \cdot 2 + 1 \cdot 4 + 1 \cdot 4 = 23$ бита против 88.

Коды не обязаны совпасть с ручной таблицей: нули и единицы на рёбрах свободны, длины же те же. В файл дерево не пишут: хранят таблицу длин, а слова восстанавливают каноническим правилом.

Упаковка шаров в двоичном кубе

Теорема 6 (Граница Хэмминга)

Если код исправляет t ошибок, то

$$|C| \cdot \sum_{i=0}^{t} \binom{n}{i} \leq 2^n.$$

Доказательство (подсчётом строк)

Шар радиуса t вокруг слова c — все строки на расстоянии не больше t от c, их $\sum_{i=0}^{t} \binom{n}{i}$ по числу способов выбрать i испорченные позиции.

Шары вокруг разных кодовых слов не пересекаются: общая строка давала бы $d \leq 2t$, хотя исправление t ошибок требует $d \geq 2t + 1$. Все шары лежат в кубе из 2^n строк — суммарный объём не превосходит 2^n.

Совершенный код

Пример

Для кода Хэмминга $(7, 4)$ объём шара радиуса 1 равен $1 + 7 = 8$.

$$16 \cdot 8 = 128 = 2^7.$$

Шары покрывают всё пространство без зазоров.

Код, достигающий границы Хэмминга, называется **совершенным**.

Код Хэмминга $(7, 4)$ — совершенный.

Нетривиальных совершенных кодов мало: коды Хэмминга и бинарный код Голея $(23, 12, 7)$.

Для остальных параметров шары не сомкнутся — между ними останутся зазоры.

Энтропия

Определение 5 (Энтропия)

Для символов с вероятностями $p_1, \ldots, p_n$:

$$H = -\sum_{i=1}^{n} p_i \log_2 p_i.$$

Пример (Энтропия на числах)

Равновероятная пара: оба слагаемых равны 0.5, и $H = 1$ бит. Достоверный символ: $\log_2 1 = 0$ — сообщать не о чем.

Частоты примера с пятью символами: $H = 0.40 \cdot 1.322 + 0.25 \cdot 2 + 0.20 \cdot 2.322 + 0.10 \cdot 3.322 + 0.05 \cdot 4.322 \approx 2.04$ бита на символ — код с $L = 2.10$ отстаёт от предела на шесть сотых.

Энтропия: честные числа

Пример (Кривая монета)

$$H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.469,$$

то есть 0.47 бита на символ: весь запас информации — в редком десятом.

Пример (Код на парах символов)

Пары источника имеют частоты $0.81, 0.09, 0.09, 0.01$.

Хаффман на них даёт $00 \to 1$, $10 \to 00$, $01 \to 011$, $11 \to 010$.

$$L = 0.81 \cdot 1 + 0.09 \cdot 2 + 0.09 \cdot 3 + 0.01 \cdot 3 = 1.29$$

бита на пару — 0.645 на символ.

Одиночный код тратит бит вместо 0.47: блоки помогают, но таблица растёт как 2^n.

Граница энтропии

Теорема 7 (Теорема о кодировании источника)

Для частот p_i и средней длины L любого префиксного кода

$$H \le L < H + 1.$$

Доказательство (оценками по отдельности)

Нижняя. Для $q_i = 2^{-l_i}$ разность $L - H = \sum_i p_i \log_2(p_i/q_i)$ неотрицательна.

По неравенству о сумме логарифмов она не меньше $\log_2\left(1/\sum_i q_i\right) \ge 0$, а $\sum_i q_i \le 1$ — это неравенство Крафта.

Верхняя. Код с $l_i = \lceil -\log_2 p_i \rceil$ префиксен: $2^{-l_i} \le p_i$.

Каждое $l_i < -\log_2 p_i + 1$, поэтому $L < H + 1$, а оптимальный код Хаффмана не хуже.

Граница: числа и блоки

Пример

Для таблицы шести символов выше $H \approx 2.42 \le L = 2.45 < 3.42$ — код в трёх сотых бита от нижнего предела.

Равенство $L = H$ возможно, только когда все p_i — степени $\frac{1}{2}$: тогда $-\log_2 p_i$ целы и дерево заполняется без остатка.

На блоках длины n граница превращается в $H \le L < H + \frac{1}{n}$: зазор тает с ростом блока.

Именно поэтому блочные коды и арифметическое сжатие приближаются к энтропии сколь угодно близко.

Пропускная способность канала

Канал передаёт биты и с вероятностью p переворачивает каждый независимо — двоичный симметричный канал.

Определение 6 (Пропускная способность)

$$C = 1 - H(p) = 1 + p\log_2 p + (1-p)\log_2(1-p).$$

p	C	**Канал**
0	1	без ошибок
0.1	0.53	умеренный шум
0.2	0.28	шум заметен
0.5	0	бесполезен

При $p = 0.5$ принятый бит не зависит от переданного — канал не несёт ничего.

Теорема Шеннона

Теорема 8 (Теорема Шеннона о кодировании (1948))

При любой скорости $R < C$ существуют коды с вероятностью ошибки, стремящейся к нулю.

При любой скорости $R > C$ надёжная передача невозможна.

Доказательство неконструктивно: случайный код большой длины хорош с высокой вероятностью, но из доказательства не извлечь сам код.

Построение явных кодов у границы — программа на семь десятилетий: Рид—Соломон, турбо-коды, LDPC, полярные коды.

Сжать, потом защитить

Защита и сжатие двигают одну величину — долю полезного в переданном.

Данные сжимают кодом Хаффмана, затем защищают кодом Хэмминга.

Сжатие убирает лишнюю избыточность, канальный код добавляет её назад — дозированно и там, где её ждёт декодер.

Порядок важен: сначала сжатие, потом защита.

Если первым поставить код Хэмминга, он вставит проверочные биты в несжатый текст, и сжимать придётся уже перемешанные с избыточностью данные — выигрыша не будет.

Обзор: циклические коды

Определение 7 (Циклический код)

Линейный код, замкнутый относительно циклического сдвига:

$$(c_0, ..., c_{n-1}) \to (c_{n-1}, c_0, ..., c_{n-2}).$$

Слово — многочлен $c(x) = c_0 + c_1 x + ... + c_{n-1} x^{n-1}$, а сдвиг — умножение на x по модулю $x^n - 1$.

Кодовые слова — в точности кратные порождающего многочлена $g(x)$, который делит $x^n - 1$.

Деление с остатком

Пример

Для кода $(7, 4)$: $g(x) = 1 + x + x^3$.

Проверка над $\mathrm{GF}(2)$:

$$x^7 - 1 = (1 + x + x^3)(1 + x + x^2 + x^4).$$

Удвоенные члены обнуляются: $x^i + x^i = 0$.

Кодирование: $c(x) = x^{n-k} m(x) - r(x)$, где $r(x)$ — остаток от деления на $g(x)$.

Декодирование — то же деление: ненулевой остаток — сигнал об ошибке.

CRC в Ethernet

1. Отправитель вычисляет остаток $r(x)$ от деления данных на $g(x)$ и дописывает его в конец пакета.
2. Получатель делит принятое на $g(x)$: ненулевой остаток — ошибка.
3. CRC-32 живёт в Ethernet, PNG и gzip.

CRC обнаруживает, но не исправляет: повреждённый пакет отбрасывается, а доставку берёт на себя транспортный уровень.

Пример

CRC-32 в Ethernet ловит любую пакетную ошибку длиной до 32 битов — царапина короче четырёх байтов не пройдёт незамеченной.

Более длинные пакеты тоже почти всегда видны: мимо проходит лишь доля 2^{-32}.

Обзор: коды Рида–Соломона

Определение 8 (Код Рида–Соломона)

Над полем $\mathrm{GF}(2^8)$ сообщение $(m_0, \ldots, m_{k-1})$ задаёт многочлен степени меньше k.

Кодовое слово — значения этого многочлена в $n = 255$ степенях примитивного элемента поля.

Многочлен степени меньше k однозначно восстанавливается по любым k значениям — это интерполяция.

Порождающий многочлен $g(x) = (x - \alpha)(x - \alpha^2)\cdots(x - \alpha^{2s})$ делит $x^n - 1$: RS-код — циклический с $2s = n - k$ проверочными символами.

Кодовое расстояние $d = n - k + 1$: два разных многочлена совпадают не более чем в $k - 1$ точках.

Маленький код RS

Пример

$\mathrm{RS}(7, 3, 5)$ над $\mathrm{GF}(8)$, где $8 = 2^3$.

Три символа кодируются семью, расстояние $d = 7 - 3 + 1 = 5$.

Исправляет до $\frac{7-3}{2} = 2$ символьных ошибок, хотя каждый символ — 3 бита.

Две символьные ошибки — до шести испорченных битов: алгебра многочлена чинит их одной операцией над символами.

Пакетные ошибки

Царапина портит биты подряд.

Код Рида–Соломона считает ошибки символами: байт испорчен целиком или нет, и до $\frac{n-k}{2}$ испорченных символов исправляются.

Пример

RS$(255, 239)$: скорость $R = \frac{239}{255} \approx 0.94$.

Избыточность всего 6% — и до 8 байтовых ошибок исправляются.

RS-коды на практике

RS-коды выбирают там, где ошибки приходят пачками, а не поодиночке: исправление идёт символами.

Царапина стирает целый участок: побитовый код потратил бы исправления на один символ, а символьный чинит его целиком.

- QR-коды — RS над $\mathrm{GF}(256)$ с уровнями коррекции от 7% до 30% повреждённой площади.
- CD и DVD — каскад CIRC: два слоя RS с перемежением против царапин и пыли.
- Цифровое телевидение DVB — $\mathrm{RS}(204, 188)$: до 8 байтовых ошибок на блок.
- RAID 6 — два проверочных диска: данные выживают при отказе двух дисков разом.

Итоги: сжатие

Линия сжатия идёт от запрета на префиксы до теоремы о кодировании источника. Частотный резерв забирает Хаффман, повторения — словарь LZ, остаток округления — арифметический код. Пределы названы точно: Крафт для длин, энтропия для частот.

- Код Хаффмана строит оптимальный префиксный код: частым символам — короткие слова.
- Префиксность даёт декодирование без задержки, неравенство Крафта — допустимые длины.
- Граница $H \le L < H + 1$ — предел частотного сжатия, счётный аргумент — несжимаемость большинства строк.
- Словарь LZ заменяет повторения ссылками, DEFLATE — это LZ77 плюс Хаффман.

Итоги: защита от ошибок

Коды с исправлением идут от границы упаковки к канальному пределу, а от него к алгебре многочленов. Граница Хэмминга говорит, сколько слов можно упаковать, а теорема Шеннона — какая скорость надёжна. Многочлены дают конструкции: от проверки пакетов до восстановления данных с исцарапанного диска.

- Граница Хэмминга ограничивает коды упаковкой шаров, $(7, 4)$ — совершенный.
- Энтропия — минимум битов на символ, теорема Шеннона делит скорости на возможные и невозможные.
- Циклические коды — алгебра многочленов: CRC в Ethernet ищет ошибки делением на $g(x)$.
- Рид—Соломон чинит пакетные ошибки символами: QR-коды, диски, RAID.

Проверьте себя: сжатие

Вопросы охватывают построение кодов и словарные схемы.

Вопросы для самопроверки

1. Постройте код Хаффмана для $(0.4, 0.3, 0.2, 0.1)$ и посчитайте среднюю длину.
2. Почему код $\{0, 10, 110, 111\}$ префиксный, а $\{0, 01, 011\}$ — нет?
3. Проверьте неравенство Крафта для $(1, 2, 3, 3, 3)$ и постройте код для $(2, 2, 3, 3)$.
4. Для источника $(0.9, 0.1)$ код Хаффмана тратит бит при энтропии 0.47 — почему?
5. Сожмите «ABRACADABRA» тройками LZ77: какая тройка кодирует финальное «ABRA»?

Проверьте себя: границы и защита

Вопросы о пределах сжатия и о кодах с исправлением ошибок.

Вопросы для самопроверки

1. Какую долю строк можно сжать на 10 битов, и почему gzip этому не противоречит?
2. Посчитайте энтропию распределения $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)$.
3. Оцените границей Хэмминга число слов кода длины 9, исправляющего две ошибки.
4. Проверьте, что $g(x) = 1 + x + x^3$ делит $x^7 - 1$ над $\mathrm{GF}(2)$.
5. Сколько байтовых ошибок исправляет $\mathrm{RS}(255, 239)$ и почему он хорош против царапины на диске?

Семестр 1: что мы прошли?

Семестр шёл от языка описания к границам вычислимого и передаваемого. Логика и множества дали язык, отношения, порядок, функции и мощность — словарь структур и первую встречу с бесконечностью. Формальная логика добавила вывод и семантику, а булева алгебра и схемы повели от формул к железу. Коды превратили избыточность в математику: расстояние, код Хэмминга, код Хаффмана, границы Шеннона. Во втором семестре — графы, автоматы, машины Тьюринга и комбинаторика.

- Чтение к экзамену — m13: «Код Хаффмана», «Словарное сжатие: LZ77 и LZ78», «Граница Шеннона», «Циклические коды», «Коды Рида–Соломона».
- Последние занятия — 22–23 декабря.
- Экзамен за семестр 1 — в зимнюю сессию (январь). Даты задаёт расписание вуза.